

CHAPTER 5

ADVERSARIAL SEARCH AND GAMES

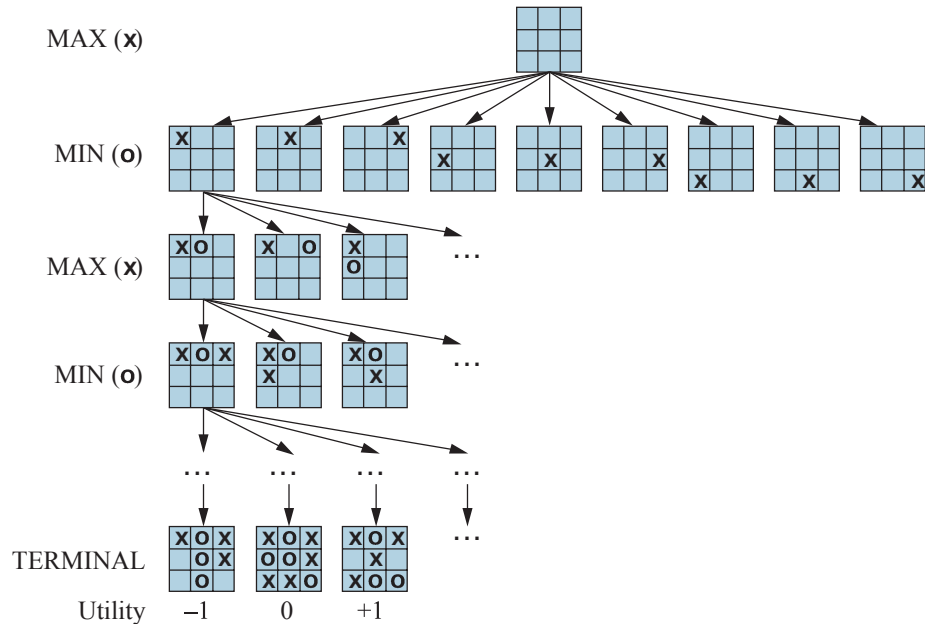


Figure 5.1 A (partial) game tree for the game of tic-tac-toe. The top node is the initial state, and MAX moves first, placing an X in an empty square. We show part of the tree, giving alternating moves by MIN (O) and MAX (X), until we eventually reach terminal states, which can be assigned utilities according to the rules of the game.

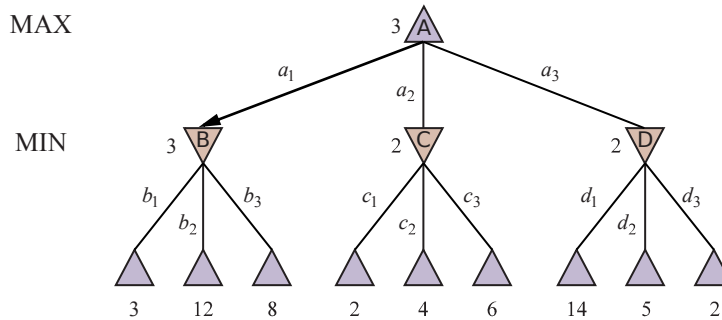


Figure 5.2 A two-ply game tree. The $\triangle$ nodes are “MAX nodes,” in which it is MAX’s turn to move, and the ∇ nodes are “MIN nodes.” The terminal nodes show the utility values for MAX; the other nodes are labeled with their minimax values. MAX’s best move at the root is a_1 , because it leads to the state with the highest minimax value, and MIN’s best reply is b_1 , because it leads to the state with the lowest minimax value.

function MINIMAX-SEARCH(*game*, *state*) **returns** an action

 player $\leftarrow$ game.TO-MOVE(*state*)

 value, move $\leftarrow$ MAX-VALUE(*game*, *state*)

return move

function MAX-VALUE(*game*, *state*) **returns** a (utility, move) pair

if game.IS-TERMINAL(*state*) **then return** game.UTILITY(*state*, player), null

$v \leftarrow -\infty$

for each *a* **in** game.ACTIONS(*state*) **do**

$v2, a2 \leftarrow$ MIN-VALUE(*game*, game.RESULT(*state*, *a*))

if $v2 > v$ **then**

$v, move \leftarrow v2, a$

return $v, move$

function MIN-VALUE(*game*, *state*) **returns** a (utility, move) pair

if game.IS-TERMINAL(*state*) **then return** game.UTILITY(*state*, player), null

$v \leftarrow +\infty$

for each *a* **in** game.ACTIONS(*state*) **do**

$v2, a2 \leftarrow$ MAX-VALUE(*game*, game.RESULT(*state*, *a*))

if $v2 < v$ **then**

$v, move \leftarrow v2, a$

return $v, move$

Figure 5.3 An algorithm for calculating the optimal move using minimax—the move that leads to a terminal state with maximum utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state and the move to get there.

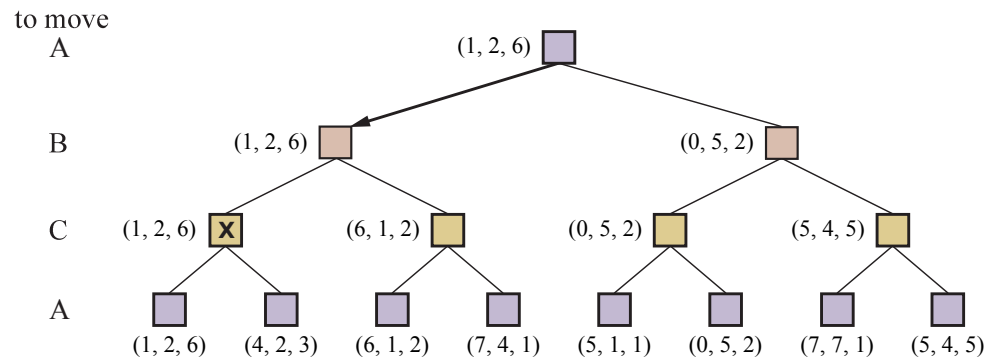


Figure 5.4 The first three ply of a game tree with three players (A , B , C). Each node is labeled with values from the viewpoint of each player. The best move is marked at the root.

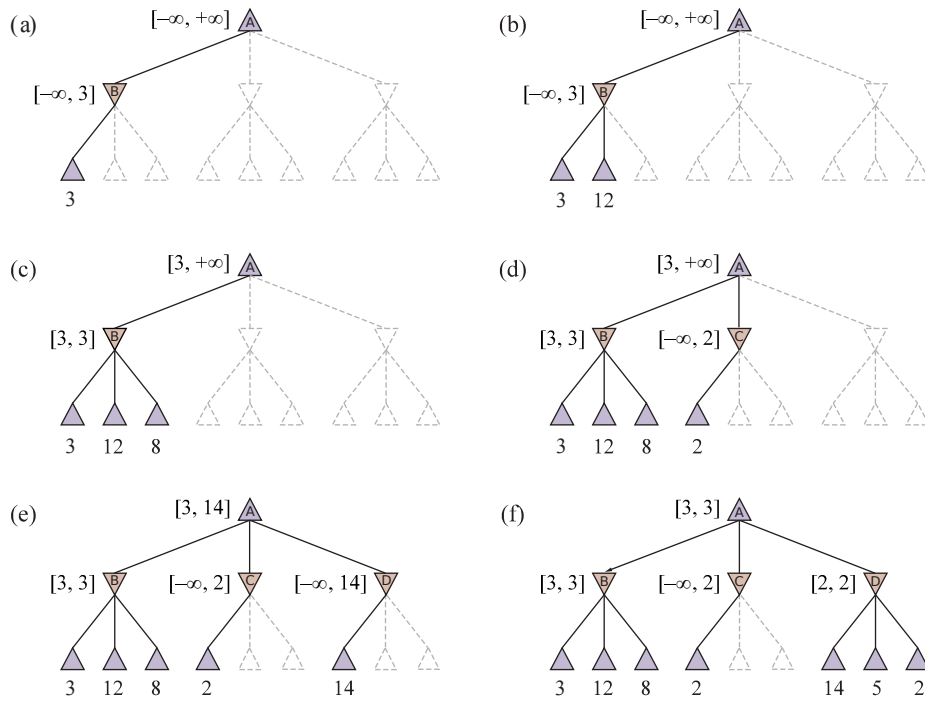


Figure 5.5 Stages in the calculation of the optimal decision for the game tree in Figure ?? . At each point, we show the range of possible values for each node. (a) The first leaf below B has the value 3. Hence, B , which is a MIN node, has a value of *at most* 3. (b) The second leaf below B has a value of 12; MIN would avoid this move, so the value of B is still at most 3. (c) The third leaf below B has a value of 8; we have seen all B 's successor states, so the value of B is exactly 3. Now we can infer that the value of the root is *at least* 3, because MAX has a choice worth 3 at the root. (d) The first leaf below C has the value 2. Hence, C , which is a MIN node, has a value of *at most* 2. But we know that B is worth 3, so MAX would never choose C . Therefore, there is no point in looking at the other successor states of C . This is an example of alpha-beta pruning. (e) The first leaf below D has the value 14, so D is worth *at most* 14. This is still higher than MAX's best alternative (i.e., 3), so we need to keep exploring D 's successor states. Notice also that we now have bounds on all of the successors of the root, so the root's value is also at most 14. (f) The second successor of D is worth 5, so again we need to keep exploring. The third successor is worth 2, so now D is worth exactly 2. MAX's decision at the root is to move to B , giving a value of 3.

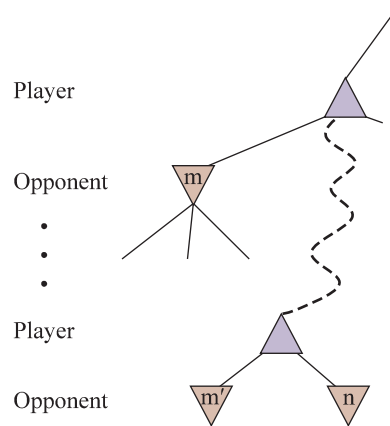


Figure 5.6 The general case for alpha-beta pruning. If m or m' is better than n for Player, we will never get to n in play.

```

function ALPHA-BETA-SEARCH(game, state) returns an action
  player  $\leftarrow$  game.TO-MOVE(state)
  value, move  $\leftarrow$  MAX-VALUE(game, state,  $-\infty$ ,  $+\infty$ )
  return move

function MAX-VALUE(game, state,  $\alpha$ ,  $\beta$ ) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v  $\leftarrow$   $-\infty$ 
  for each a in game.ACTIONS(state) do
    v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )
    if v2 > v then
      v, move  $\leftarrow$  v2, a
       $\alpha \leftarrow$  MAX( $\alpha$ , v)
    if v  $\geq$   $\beta$  then return v, move
  return v, move

function MIN-VALUE(game, state,  $\alpha$ ,  $\beta$ ) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v  $\leftarrow$   $+\infty$ 
  for each a in game.ACTIONS(state) do
    v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )
    if v2 < v then
      v, move  $\leftarrow$  v2, a
       $\beta \leftarrow$  MIN( $\beta$ , v)
    if v  $\leq$   $\alpha$  then return v, move
  return v, move

```

Figure 5.7 The alpha-beta search algorithm. Notice that these functions are the same as the MINIMAX-SEARCH functions in Figure ??, except that we maintain bounds in the variables α and β , and use them to cut off search when a value is outside the bounds.

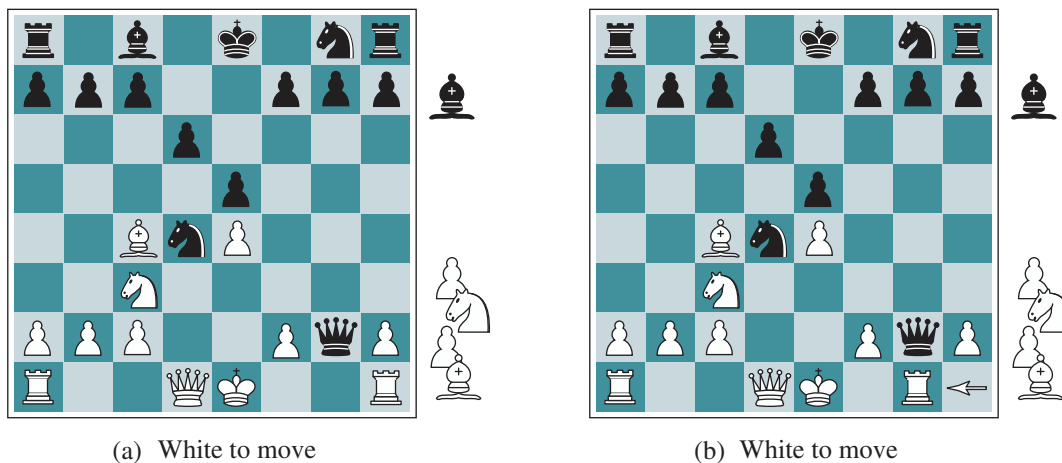


Figure 5.8 Two chess positions that differ only in the position of the rook at lower right. In (a), Black has an advantage of a knight and two pawns, which should be enough to win the game. In (b), White will capture the queen, giving it an advantage that should be strong enough to win.

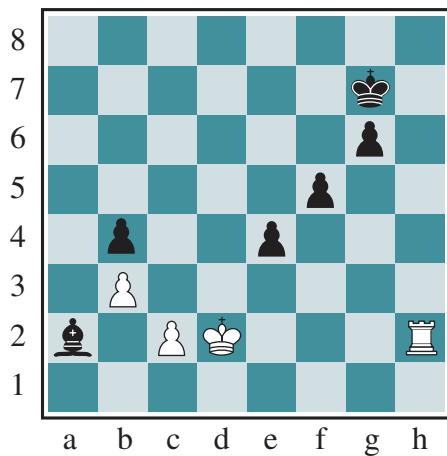


Figure 5.9 The horizon effect. With Black to move, the black bishop is surely doomed. But Black can forestall that event by checking the white king with its pawns, encouraging the king to capture the pawns. This pushes the inevitable loss of the bishop over the horizon, and thus the pawn sacrifices are seen by the search algorithm as good moves rather than bad ones.

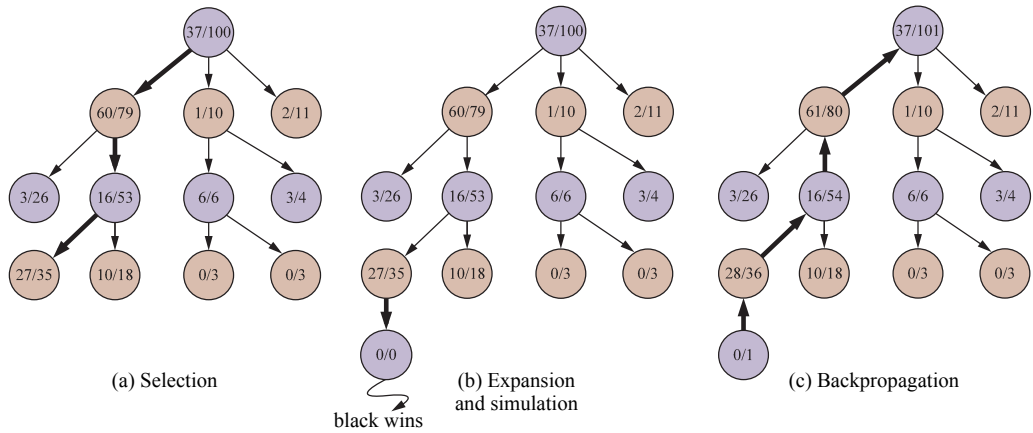


Figure 5.10 One iteration of the process of choosing a move with Monte Carlo tree search (MCTS) using the upper confidence bounds applied to trees (UCT) selection metric, shown after 100 iterations have already been done. In (a) we select moves, all the way down the tree, ending at the leaf node marked 27/35 (for 27 wins for black out of 35 playouts). In (b) we expand the selected node and do a simulation (playout), which ends in a win for black. In (c), the results of the simulation are back-propagated up the tree.

```

function MONTE-CARLO-TREE-SEARCH(state) returns an action
  tree  $\leftarrow$  NODE(state)
  while IS-TIME-REMAINING() do
    leaf  $\leftarrow$  SELECT(tree)
    child  $\leftarrow$  EXPAND(leaf)
    result  $\leftarrow$  SIMULATE(child)
    BACK-PROPAGATE(result, child)
  return the move in ACTIONS(state) whose node has highest number of playouts

```

Figure 5.11 The Monte Carlo tree search algorithm. A game tree, *tree*, is initialized, and then we repeat a cycle of SELECT / EXPAND / SIMULATE / BACK-PROPAGATE until we run out of time, and return the move that led to the node with the highest number of playouts.

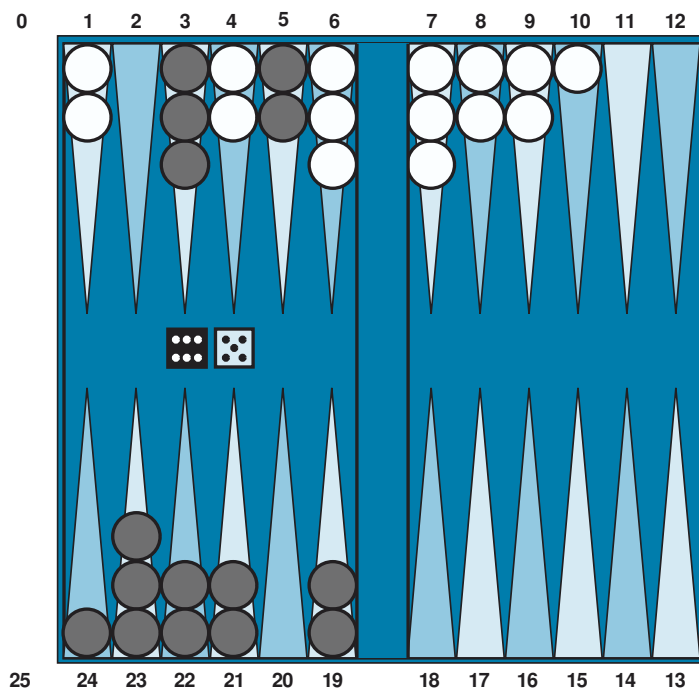


Figure 5.12 A typical backgammon position. The goal of the game is to move all one's pieces off the board. Black moves clockwise toward 25, and White moves counterclockwise toward 0. A piece can move to any position unless multiple opponent pieces are there; if there is one opponent, it is captured and must start over. In the position shown, Black has rolled 6-5 and must choose among four legal moves: (5-11,5-10), (5-11,19-24), (5-10,10-16), and (5-11,11-16), where the notation (5-11,11-16) means move one piece from position 5 to 11, and then move a piece from 11 to 16.

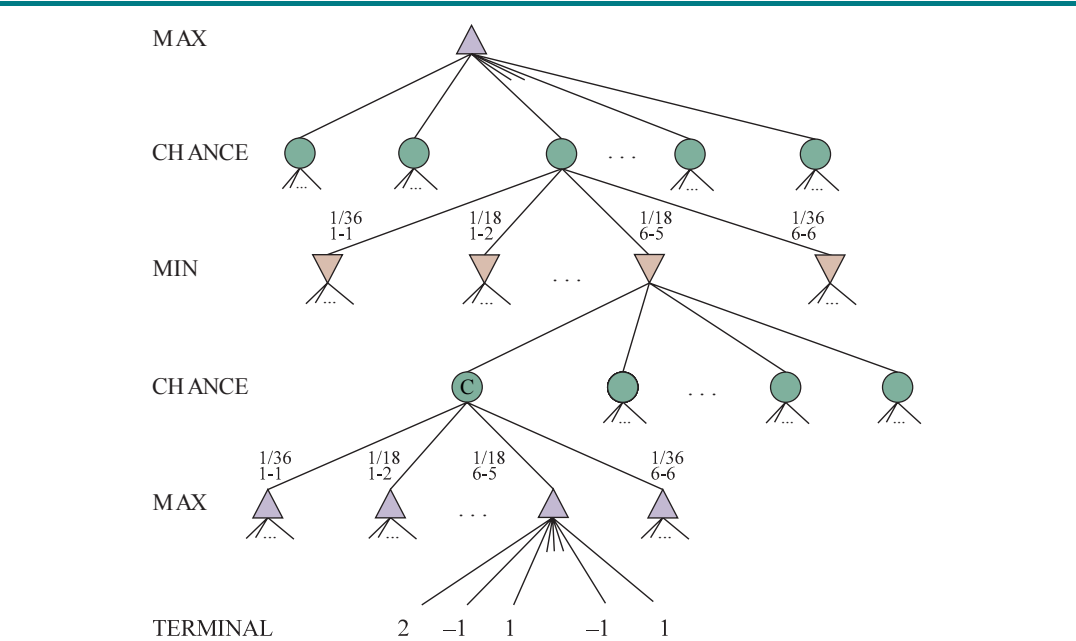


Figure 5.13 Schematic game tree for a backgammon position.

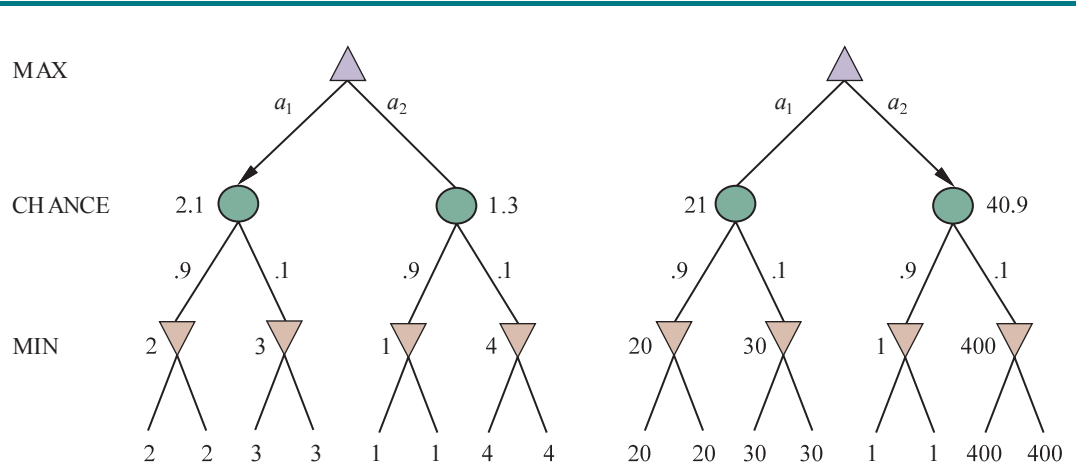


Figure 5.14 An order-preserving transformation on leaf values changes the best move.

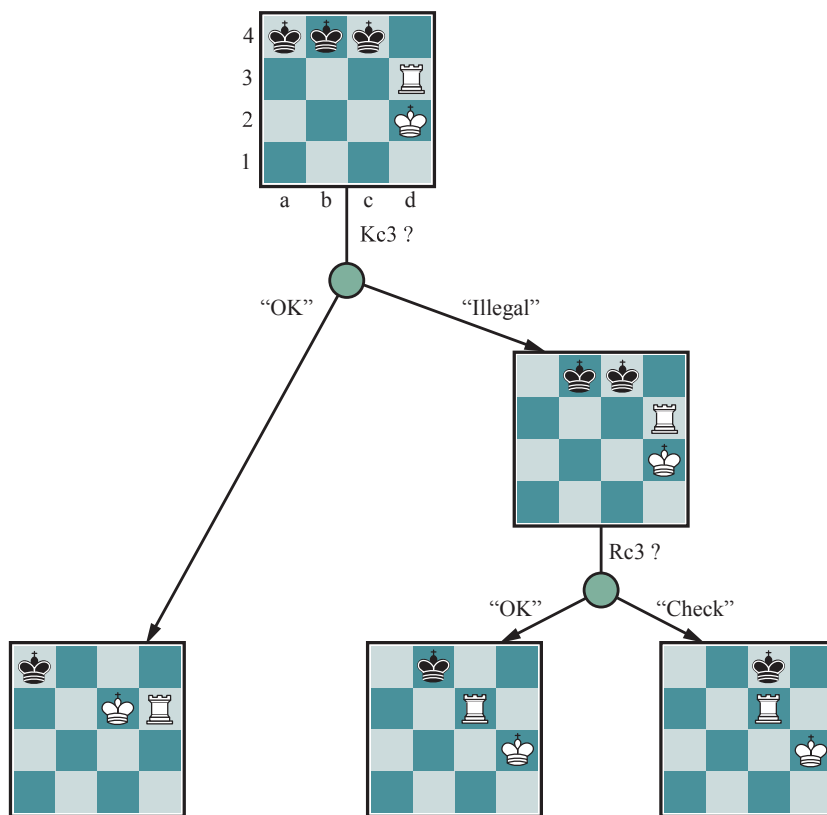


Figure 5.15 Part of a guaranteed checkmate in the KRK endgame, shown on a reduced board. In the initial belief state, Black's king is in one of three possible locations. By a combination of probing moves, the strategy narrows this down to one. Completion of the checkmate is left as an exercise.

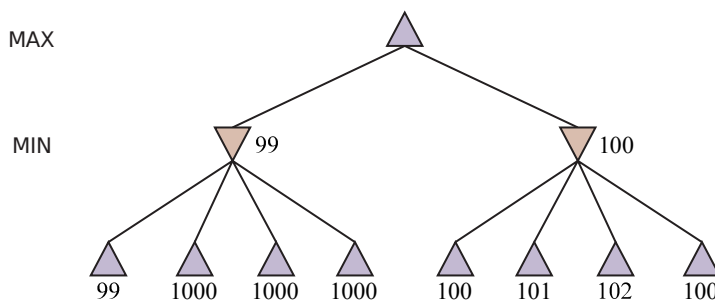


Figure 5.16 A two-ply game tree for which heuristic minimax may make an error.